Expanded Background and Related Work for OmniFlow

Status. This file is a standalone LaTeX report rendered with the existing paper template, bibliography, and ACM reference style. It is writing material for the paper's Background and Related Work sections, not an active replacement for `background.tex` or `related.tex`. External numbers below are reported by the cited papers and are not OmniFlow evaluation results.

1 Background: Mobile GUI Agents as Repeated Observe-Reason-Act Systems

Recent mobile GUI agents usually execute tasks through an observe-reason-act loop. At each step, the system observes the current screen through a screenshot, an accessibility tree, or both; constructs a prompt or structured state; asks an LLM or VLM to choose the next action; dispatches a tap, swipe, text input, back action, or app-level command; and repeats after the environment changes. AndroidWorld formalizes this setting as a dynamic Android benchmark with programmatic initialization, success checking, and teardown logic for 116 tasks across 20 real-world Android apps [12]. Android in the Wild (AITW) provides a much larger demonstration corpus with 715K episodes, around 30K unique instructions, Android versions 10–13, and eight Pixel device types [13]. These benchmarks make two facts clear. First, GUI agents need semantic interpretation of natural language and visual state. Second, the execution substrate is heterogeneous: device size, OS version, app state, account data, and available widget metadata all affect whether a recorded action can still be used.

The cost profile of this loop is important for systems work. A step-wise agent pays model latency and token cost every time it asks the model to select the next action. V-Droid reports that prior high-performing agents on AndroidWorld can take tens of seconds per step, while its verifier-based design reaches 4.3 s per step and 59.5% task success on AndroidWorld [2]. The key point for OmniFlow is that this is still per-step online decision making: the agent reduces the cost of each decision, yet a repeated routine still passes through the model or verifier for each screen transition. AndroidWorld also reports that its best baseline in the original benchmark completed 30.6% of tasks, leaving significant headroom for robust mobile control [12]. In practice, even a fast per-step policy creates cumulative cost for long workflows, especially when the same or similar routines are repeated by many users.

Repeated mobile behavior is not a corner case. Smartphone-use studies describe brief, repetitive checking behavior as a common usage pattern [10]. App launching itself is sufficiently predictable that FALCON used location and temporal access patterns to prelaunch likely apps and reduce perceived launch delay [17]. AITW's 715K episodes over roughly 30K unique instructions further suggests that device-control workloads contain repeated task forms even when the concrete screen state and parameters vary [13]. This motivates a cache-system view of GUI agents. If a user, device, or prior execution has already produced a verified route through a repeated GUI segment, later executions should be able to reuse that segment without asking the online model to reinterpret every screen.

The challenge is that GUI reuse is harder than replaying an API call. Android accessibility trees expose bounds and hierarchy reliably when elements are visible, while semantic fields such as `resource-id`, `text`, and `content-desc` depend on developer annotation, framework behavior, and app implementation. WebView, custom-rendered views, obfuscated identifiers, dynamic list content, dialogs, IME overlays, and account-specific state make raw coordinate replay and exact selector replay fragile. A UI transition graph (UTG) captures useful structure: nodes represent GUI states and edges represent actions or transitions. However, a graph built from raw XML snapshots or static window transitions records possible movement through an interface; it does not by itself decide whether a historical action has the same task effect in the current state. For OmniFlow, the reusable object therefore has to combine state localization, target transfer, and effect checking. The paper's current abstraction is a `Function`: a reusable executable task segment tied to a start condition, action evidence, transfer logic, and terminal/effect verification. `PageVectorSet` provides a privacy-bounded state key for candidate recall, and `run_log -> function asset` turns successful executions into future cache entries.

2 Related Work

2.1 Record-and-Replay and Programming by Demonstration

Classic mobile automation systems show the value of executable reuse. RERAN records low-level Android input events and replays timing-sensitive gestures and sensor events [4]. Its strength is fidelity: it captures the real event stream rather than a symbolic approximation of user input. This makes it valuable for reproducing executions during testing or debugging. Its limitation follows from the same design. The replay is tied to the recorded device state, timing, layout, and app behavior. When an element shifts, a dialog appears, or a list

item moves, replaying the same event stream can target the wrong location or fail to produce the intended task effect.

SUGILITE takes a higher-level programming-by-demonstration approach for smartphone automation [7]. A user demonstrates a task directly in Android apps, and the system uses the accessibility API to construct an automation script that can later be invoked through natural language. This line of work is important because it treats repeated GUI work as something that users should not have to reperform manually. It also shows a limitation that remains central for GUI-agent cache systems: demonstrations must be generalized from observed widgets and states to future states. If the future UI lacks stable attributes or appears in a different layout/context, the demonstration needs repair, clarification, or a fallback path.

DroidBot and related GUI-testing tools construct state-transition models during exploration [8]. DroidBot can generate a UTG from UI-guided testing and use that graph to guide future exploration. Static analysis work such as GATOR's window transition graph estimates possible Android GUI transitions from application code [18]. These systems provide graph structure, but their objective is testing or static modeling rather than executing user goals from natural language under dynamic state. Static graphs can over-approximate infeasible transitions or miss runtime-only transitions; dynamic exploration graphs depend on coverage and collected app states. OmniFlow uses a graph vocabulary because state and transition structure are useful, while the cache object is a verified `Function` with start-node preconditions and terminal/effect checks.

2.2 Memory-Based Mobile GUI Agents

AutoDroid is a representative memory-augmented mobile GUI agent [15]. Its method combines LLM commonsense with app-specific knowledge collected through automated dynamic analysis. The system builds a functionality-aware UI representation, injects exploration-based memory to augment the LLM with app knowledge, and uses multi-granularity query optimization to reduce inference cost. It reports 90.9% action accuracy and 71.3% task success on a benchmark of 158 common tasks, outperforming GPT-4-powered baselines by 36.4% in action accuracy and 39.7% in task success. AutoDroid therefore demonstrates that app memory and structured UI representation can materially improve task automation.

The limitation for OmniFlow's target problem is the execution boundary. AutoDroid still treats memory mainly as knowledge for the LLM's decision process. The memory helps the model understand app-specific states and actions, but a repeated task is not necessarily served as a directly executable cache hit that skips online reasoning for the matched segment. The app-specific exploration phase also creates a cold-start cost and a deployment question: users or hosts need a way to collect, store, and update app memory before reuse becomes available. For a cache-first system, the reusable unit has to be callable by the execution runtime, guarded by state/effect evidence, and written back from accepted run logs instead of existing only as prompt context.

MobileGPT makes the reuse goal more explicit through a human-like app memory pipeline [6]. It frames mobile task learning as explore, select, derive, and recall. The agent explores an app, selects relevant knowledge, derives reusable subtask procedures, and recalls them for later tasks. It evaluates on 185 tasks across 18 apps and reports 82.7% accuracy for automating and learning new tasks, 98.75% adaptation accuracy across contexts, and latency/cost reductions of 62.5% and 68.8% relative to a GPT-4 baseline. This is strong evidence that memory over prior task experience can reduce repeated cost.

MobileGPT's limitation is also instructive. A human-like memory stores procedures and app knowledge that the LLM can recall and adapt. The online agent still needs to bind recalled knowledge to the current screen, and the memory is not the same object as a guarded executable cache entry. When UI attributes are missing, list content changes, or app state diverges from the learned procedure, the system has to adapt through the current reasoning loop. OmniFlow's `Function` boundary targets a narrower but more directly executable form of reuse: if the current state satisfies the start precondition and transfer checks, the runtime executes the cached segment under `control-aware execution`; if checks fail, the host agent remains the fallback.

AppAgent uses autonomous exploration or human demonstrations to build an app knowledge base for a multimodal smartphone agent [19]. It operates through a simplified action space that mimics user interactions such as taps and swipes, and it was tested on 50 tasks across 10 applications. AppAgent is relevant because it shows that exploration documents and demonstrations can help a general multimodal agent operate unfamiliar apps. Its reuse unit, however, remains knowledge for future decisions. It does not define a shared, privacy-bounded `PageVectorSet` state key or a `run_log -> function asset` lifecycle in which verified executions become executable cache assets.

2.3 Programmatic and State-Machine GUI Agents

Programmatic GUI agents move beyond prompting by converting task knowledge into executable code or state-machine plans. AutoDroid-V2 adopts a full task-level script generation approach for mobile GUI agents [16]. Its motivation is to make small-language-model GUI agents more effective by generating code that can execute task logic. This direction is useful for repeated workflows because executable scripts can reduce repeated natural-language reasoning. The tradeoff is that task-level scripts need a stable enough app model, current-state alignment, and error handling when the live UI deviates from the generated assumptions. A complete task

script is also a relatively coarse reuse object: if the beginning of a workflow changes while the effect-carrying middle segment is still valid, the script needs repair or regeneration.

ActionEngine is a recent state-machine-memory system that shifts GUI agents from reactive step-by-step operation to programmatic execution [20]. It uses a Crawling Agent to construct an updatable state-machine memory through offline exploration and an Execution Agent to synthesize complete Python programs for online task execution. On Reddit tasks from WebArena, it reports 95% task success with about one LLM call on average, 11.8× cost reduction, and 2× end-to-end latency reduction compared with a strong vision-only baseline. ActionEngine is close to OmniFlow in recognizing that a GUI can be reused through a state-machine-like memory rather than reinterpreted at every step.

The boundary remains different. ActionEngine's reported result is a web task setting, and the method depends on an offline crawler building a state-machine memory from the target GUI. Execution failures trigger vision-based regrounding and memory updates. OmniFlow's mobile setting has stronger device heterogeneity, privacy, and app-state constraints. The paper's current contribution should therefore avoid claiming a broad superiority over state-machine agents. The precise gap is that OmniFlow makes the cache unit a `Function` derived from accepted run logs, uses `PageVectorSet` as the privacy-bounded state-recall object, and requires start-node, route-only bridge, action transfer, and terminal/effect verification before a cache hit is counted as successful.

AgentProg addresses long-horizon GUI tasks from the context-management angle [14]. It represents interaction history as a program with variables and control flow, and it adds a global belief state to handle partial observability and environmental change. This is important because long GUI tasks quickly accumulate history that can overflow or dilute model context. AgentProg's limitation relative to OmniFlow is that it optimizes what the online agent retains and reasons over, while OmniFlow aims to bypass online reasoning for already matched `Function` segments. Program-guided context management and cache-first execution are complementary: a host agent can use program-style memory for misses and long-horizon fallback, while OmniFlow serves verified repeated segments as function-cache hits.

2.4 Verifier-Based and Per-Step Acceleration

V-Droid changes the role of the model from generator to verifier [2]. Instead of asking the LLM to autoregressively produce an action, it constructs a discretized action space and asks a trained verifier to score candidate actions. The method includes prefilling-only verification for speed, pairwise progress-preference training, and a scalable human-agent annotation loop. The reported model uses Llama-3.1-8B and 110K collaboratively annotated samples across four rounds. It reports 59.5% success on AndroidWorld, 38.3% on AndroidLab, and 49% on MobileAgentBench; the paper also reports 4.3 s step-wise latency and 0.74 s decision latency in the AndroidWorld comparison table.

V-Droid is strong evidence that mobile GUI agents can be made faster and more accurate without relying on very large cloud models for every action. Its limitation for repeated-work reuse is structural: a verifier still evaluates candidates on each step. A cache-first system asks a different question. For a matched repeated segment, can the system skip the online decision and execute a previously accepted `Function` directly, with only local checks and effect verification? OmniFlow should be framed as complementary to V-Droid rather than as a replacement. A verifier-based agent can serve as the host fallback for misses, ambiguous matches, unsafe transfers, or novel states.

Other per-step acceleration methods reduce the amount of visual or historical context sent into the model. ShowUI builds a vision-language-action model for GUI agents with UI-guided visual-token selection and interleaved vision-language-action streaming [9]. It reports a 2B model trained with 256K data, 75.1% zero-shot screenshot grounding accuracy, 33% redundant visual-token reduction during training, and 1.4× speedup. FocusUI targets efficient UI grounding by selecting instruction-relevant visual tokens while preserving positional continuity through PosPad [11]. It reports that FocusUI-7B improves ScreenSpot-Pro performance by 3.7% over GUI-Actor-7B and, with 30% visual-token retention, drops only 3.2% while achieving up to 1.44× faster inference and 17% lower peak GPU memory.

These methods are valuable because high-resolution UI screenshots create heavy visual-token workloads. Their limitation is that they optimize the current model call rather than removing the need for the model call on repeated segments. They help a host agent decide what to do on a screen; they do not by themselves turn a successful historical run into a callable `Function` cache entry. OmniFlow's benefit should be stated at the segment boundary: a cache hit skips online reasoning for the matched `Function`, while per-step accelerators remain useful for miss handling, fallback, and new or ambiguous states.

2.5 GUI Grounding Models

GUI grounding models address the target-localization subproblem. SeeClick trains a visual GUI agent that relies only on screenshots and introduces ScreenSpot, a grounding benchmark spanning mobile, desktop, and web environments [1]. Its key insight is that many GUI agents depend on long structured representations such as HTML or accessibility trees, while screenshots are universal across platforms. UGround similarly advocates visual-only GUI grounding and trains on 10M elements from 1.3M screenshots; it reports up to 20% absolute improvement over prior GUI visual grounding models and shows gains in downstream agent settings [5]. These

models are especially relevant when accessibility metadata is missing, noisy, or unavailable.

The limitation is runtime and problem scope. A visual grounding model receives an instruction and current screenshot, then predicts a coordinate or element. This solves the zero-prior grounding problem. OmniFlow's transfer setting has a different prior: the system already has a successful source run, a source action, neighboring anchors, and a candidate current state. The task is to rebind a recorded action to the current GUI under a cache-hit contract. A grounding model could be used as a fallback oracle when structural transfer is ambiguous, but using a VLM for every cached step would reintroduce the per-step model cost that cache-first execution is designed to avoid.

This distinction is important for fair related work. GUI grounding papers should not be described as weak baselines. They solve a broad and difficult generalization problem from screenshot and language alone. OmniFlow narrows the online problem by spending prior execution evidence: a `Function` carries source context and effect evidence, `PageVectorSet` narrows the candidate state family, and `control-aware execution` rejects or falls back when the transfer evidence is insufficient.

2.6 Benchmarks, Datasets, and What They Measure

AndroidWorld and AITW support different evaluation questions. AITW measures learning from large-scale demonstrations across device versions, screen sizes, applications, and natural-language instructions [13]. It highlights visual robustness and variation in user commands. AndroidWorld measures live task execution with programmatic success signals over 116 tasks and 20 apps [12]. It is stronger for end-to-end validation because each task has initialization and success-checking logic, but it still evaluates agents under benchmark-specific task definitions.

MobileViews contributes scale for representation and grounding analysis [3]. Recent descriptions report a million-scale mobile GUI dataset with over 1.2M unique screens from more than 30K Android apps, and earlier versions report hundreds of thousands of screenshot-view-hierarchy pairs over more than 20K apps. Such datasets are useful for studying how often UI attributes are present, how often WebView or container controls obscure semantic signals, and how representation methods behave across app families. They do not directly certify task success unless paired with action, state transition, and effect labels.

For OmniFlow, this means the evaluation narrative should keep three measurements separate. First, representation recall asks whether `PageVectorSet` retrieves the correct state family or `Function` candidates. Second, action transfer asks whether a recorded action can be safely rebound to the current GUI. Third, end-to-end success asks whether the whole task reaches a terminal/effect condition under the benchmark oracle or app-specific verifier. A high recall number alone is not a live execution guarantee, and a live success number from one bounded workload is not a production-scale network-effect result.

2.7 Positioning of OmniFlow

The related work suggests a precise gap. Record-and-replay systems provide executable reuse, but their memory is tightly bound to the recorded path and device state. Memory-based GUI agents reuse app knowledge, but much of the reuse remains prompt or procedure context for online model decisions. Programmatic agents synthesize executable plans, but they commonly depend on upfront exploration or coarse task programs that need repair when the live GUI diverges. Per-step acceleration and visual grounding make each decision cheaper or more accurate, while repeated workflows still incur online decisions.

OmniFlow's paper should state a narrow affirmative claim. It treats successful GUI executions as a cache construction source. A canonical `run_log` becomes a reusable `Function` asset. A cache lookup retrieves candidate `Functions` and compact state evidence. `PageVectorSet` provides privacy-bounded state recall without sharing raw screenshots, raw XML, or raw text as the primary shared object. On a cache hit, Flow checks the start-node precondition, optionally uses route-only navigation to reach the start state, transfers recorded actions through source-current evidence, and verifies terminal/effect evidence under `control-aware execution`. Misses, unsafe matches, ambiguous transfers, or failed checks return to the host agent or fallback path.

This positioning avoids overclaiming. OmniFlow does not need to be a new foundation GUI planner. It does not need to make first-run exploration free. It does not need to prove that `PageVectorSet` alone guarantees correct execution. The contribution is a cache-system boundary for repeated GUI work: accepted execution evidence becomes callable assets, cache hits skip online reasoning for matched `Function` segments, and shared reuse is governed by compact state representation plus verification rather than raw replay.

3 Method-Level Comparison Summary

4 Source Audit Notes

Primary or project sources checked for this material include AndroidWorld [12], AITW [13], AutoDroid [15], MobileGPT [6], AppAgent [19], V-Droid [2], ActionEngine [20], AgentProg [14], SeeClick [1], UGround [5], ShowUI [9], FocusUI [11], SUGILITE [7], RERAN [4], DroidBot [8], GATOR/WTG [18], user repetition studies [10], and FALCON [17]. Do not use unverified references from `REFERENCE_INDEX.md`, such as `hermes` or `vhm`, for precise claims until an exact source is supplied.

References

[1] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. https://arxiv.org/abs/2401.10935

[2] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).

[3] Longxi Gao, Li Zhang, Shihe Wang, Pengzhi Gao, Wei Liu, Jian Luan, Shangguang Wang, Yuanchun Li, and Mengwei Xu. 2025. Mobile-Views: A Million-scale and Diverse Mobile GUI Dataset. *arXiv preprint arXiv:2409.14337* (2025). https://arxiv.org/abs/2409.14337

[4] Lorenzo Gomez, Iulian Neamtiu, Tanzirul Azim, and Todd Millstein. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 72–81.

[5] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=kxnoqaisCT

[6] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.

[7] Toby Jia-Jun Li, Amos Azaria, and Brad A Myers. 2017. SUGILITE: creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI conference on human factors in computing systems*. 6038–6049.

[8] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2017. DroidBot: A Lightweight UI-Guided Test Input Generator for Android. In *Proceedings of the 39th International Conference on Software Engineering Companion*. 23–26. doi:10.1109/ICSE-C.2017.8

[9] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.

[10] Antti Oulasvirta, Tye Rattenbury, Lingyi Ma, and Eeva Raita. 2012. Habits make smartphone use more pervasive. *Personal and Ubiquitous computing* 16, 1 (2012), 105–114.

[11] Mingyu Ouyang, Kevin Qinghong Lin, Mike Zheng Shou, and Hwee Tou Ng. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928* (2026).

[12] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=il5yUQsrjC

[13] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. 2023. Android in the wild: A large-scale dataset for android device control. *arXiv preprint arXiv:2307.10088* (2023).

[14] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).

[15] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th annual international conference on Mobile computing and networking*. 543–557.

[16] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.

[17] Tingxin Yan, David Chu, Deepak Ganesan, Aman Kansal, and Jie Liu. 2012. Fast app launching for mobile devices using predictive user context. In *Proceedings of the 10th international conference on Mobile systems, applications, and services*. 113–126.

[18] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. 2015. Static Window Transition Graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering*. 658–668. doi:10.1109/ASE.2015.76

[19] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–20.

[20] Hongbin Zhong, Fazle Faisal, Luis França, Tanakorn Leesataporn-wongsa, Adriana Szekeres, Kexin Rong, and Suman Nath. 2026. Actionengine: From reactive to programmatic gui agents via state machine memory. *arXiv preprint arXiv:2602.20502* (2026). https://arxiv.org/abs/2602.20502

Table 1. Method-level comparison for background and related work. Metrics in this table are reported by the cited papers and are not OmniFlow evaluation results.

Direction	Representative work	Method	Reported metrics from source	Limitation for OmniFlow's target problem
Record/replay	RERAN [4]	Captures and replays low-level Android input and sensor events with timing fidelity.	Focuses on replay fidelity for Android executions.	Tied to recorded timing, device state, and layout; lacks semantic/effect verification for changed GUI state.
Programming by demonstration	SUGILITE [7]	User demonstrates an Android task; accessibility APIs support later automation from a verbal command.	Demonstrates end-user smartphone automation by demonstration.	Generalization depends on future UI attributes and state compatibility.
App-memory agent	AutoDroid [15]	Automated exploration builds app-specific memory and functionality-aware UI representation for LLM execution.	90.9% action accuracy and 71.3% task success on 158 tasks.	Memory guides online decisions; app exploration is a cold-start cost; direct executable cache-hit semantics are outside its main boundary.
Human-like memory	MobileGPT [6]	Explore, select, derive, and recall; stores modular procedures and app memory.	82.7% task learning/automation accuracy; 98.75% context adaptation; 62.5% latency and 68.8% cost reduction vs. GPT-4 baseline.	Recalled memory still needs current-state binding and online adaptation.
Knowledge-base multimodal agent	AppAgent [19]	Exploration or demonstrations build a knowledge base for a multimodal smartphone agent.	Tested on 50 tasks in 10 apps.	Knowledge base supports reasoning; it is not a verified `Function` cache store.
Programmatic state memory	ActionEngine [20]	Crawling Agent builds state-machine memory; Execution Agent synthesizes Python programs.	95% success on Reddit/WebArena tasks; about one LLM call on average; 11.8× cost reduction and 2× latency reduction.	Depends on offline crawling and program repair; reported setting is web tasks; live mobile heterogeneity and privacy-bounded sharing require separate treatment.
Long-horizon context management	AgentProg [14]	Represents interaction history as program variables/control flow with a global belief state.	Reports state-of-the-art results on AndroidWorld and extended long-horizon tasks.	Optimizes retained context for the online agent; repeated matched segments still need a cache-execution boundary to skip online reasoning.
Verifier agent	V-Droid [2]	Scores discrete candidate actions with a trained verifier and prefilling-only workflow.	59.5% AndroidWorld success; 38.3% AndroidLab; 49% MobileAgentBench; 4.3 s step latency; 0.74 s decision latency in the reported table.	Strong per-step decision method; repeated routines still involve verifier calls at each step.
Visual grounding	SeeClick and UGround [1, 5]	Screenshot-only GUI grounding to locate actionable elements.	SeeClick introduces ScreenSpot; UGround trains on 10M elements from 1.3M screenshots and reports up to 20% absolute improvement over prior GUI grounding models.	Solves zero-prior grounding; using it on every cached step reintroduces model inference.
Visual-token acceleration	ShowUI and FocusUI [9, 11]	Selects or preserves important visual tokens for faster GUI grounding/agent inference.	ShowUI reports 75.1% zero-shot grounding, 33% redundant-token reduction, and 1.4× speedup; FocusUI reports 1.44× faster inference and 17% lower peak GPU memory with 30% token retention.	Reduces model-call cost; cache-first reuse removes model calls for matched `Function` segments.